Selecting Satellite Instruments

Batch 5 - Space systems: 0/1 knapsack with real-valued masses solved by Branch and Bound

Mass limit 15.0 kg | Power limit 35 W | 6 candidate instruments

Karthikeya Prabu V K (DP) | Sree Ashwath S S (Branch and bound) | Vasantha Kumar M (Greedy)

Roadmap and who presents what

1

Karthikeya Prabu V K

Problem model, data structures, why DP and others are rejected, timing experiments

2

Sree Ashwath S S

Branch and bound: bound, pseudocode, trace, complexity, proof, 35 W change

3

Vasantha Kumar M

Greedy evaluation, search orders, worst-case input, real-world reference

1. Problem modelling

- Input: n instruments, mass m_i (real, 0.1 g precision), value v_i, capacity C = 15.0 kg
- Output: x in {0,1}^n (fully in or out)
- Objective: maximise sum v_i x_i
- Constraint: sum m_i x_i <= C
- Type: OPTIMISATION (decision form is NP-complete)

1,050,007

DP cells at 0.1 g (7 x 150,001)

105,007

DP cells in whole grams (7 x 15,001)

Q1: fine for 6 items, but cells = (n+1)(W+1) grows with the NUMBER W, so it is pseudo-polynomial.

2. Data structure analysis: the search frontier

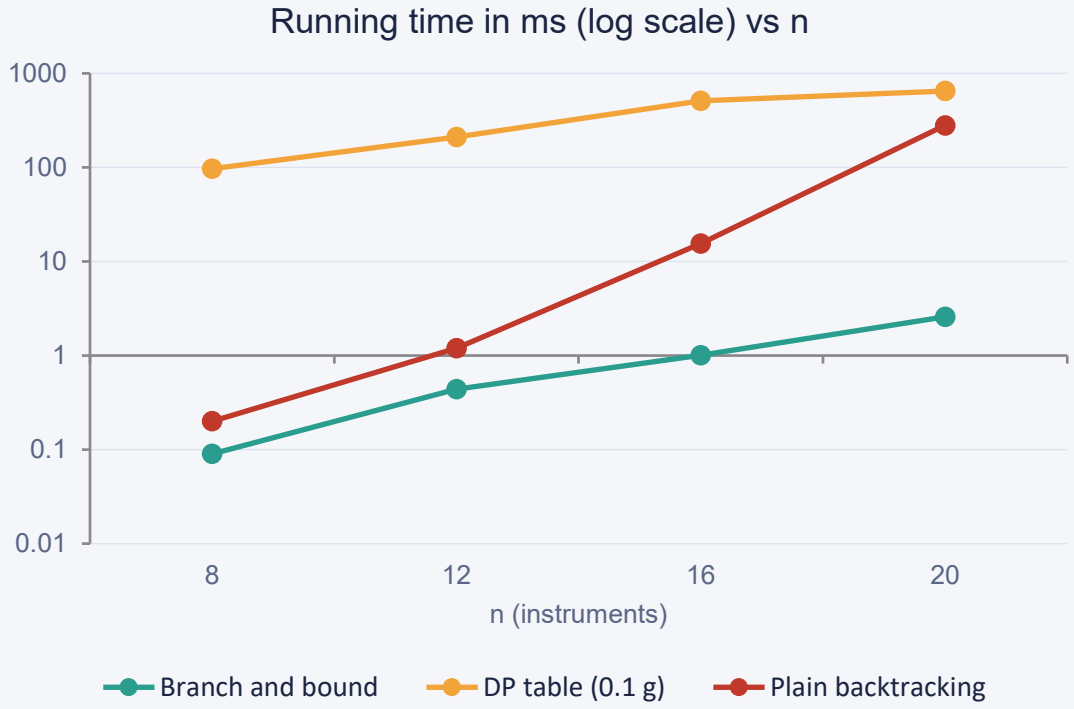
Structure	Insert	Extract	Space	On our instance
Stack (DFS)	O(1)	O(1)	O(n)	47 generated, optimum at node 42
FIFO queue (BFS)	O(1)	O(1)	up to O(2^n)	49 generated, optimum at node 24
Binary heap (best-first)	O(log F)	O(log F)	up to O(2^n)	47 generated, optimum at node 36
Items sorted by density	-	O(1) scan	O(n)	bound = O(n) scan

Choice: max-heap keyed by the bound (best-first). It always expands the most promising node and stops the moment the heap top cannot beat the incumbent. A stack is the fallback when memory is tight.

4. Why the other four techniques fail

Technique	Concrete reason on this problem
Greedy	Density order takes P1,P2,P3 = 142 (160 if it skips misfits); optimum is 174 and drops P1. Greedy-choice property fails.
Backtracking (no bound)	Visits every feasible partial selection, up to 2^(n+1)-1 nodes: 1,224,390 nodes at n = 20 vs 1,909 for B&B.
Divide and conquer	Halves share one capacity, so subproblems are not independent: the right half must be solved for C - a, for real-valued a. Combining is meet-in-the-middle, still exponential.
Dynamic programming	Substructure exists, but the state needs a 0.1 g grid: 1.05 M cells, O(nW), and a 2-D table once power is added. 0.65 s vs 0.0026 s at n = 20.

Implementation and empirical analysis



- Code: B&B plus two rejected alternatives (DP, plain backtracking).
- Five sizes timed: n = 8, 12, 16, 20, 24 (DP at n = 24: 803 ms).
- B&B grows roughly like the tree it explores, far below the c x 2^n reference; backtracking tracks 2^n.
- Replace with your own timings.csv values.

3. Technique: Branch and Bound

Bounding function

UB = v + F(k, C - m)
F = fractional knapsack optimum on the remaining items (fill by value/kg, take the last item partly).

Root: P1+P2+P3 = 142 plus 4.5/6.1 of P4 gives UB = 195.11.

Why it is valid

- Every 0/1 completion is also a fractional solution, so UB >= any completion.
- Density-order filling is optimal for the fractional problem (exchange argument).
- Values are integers, so prune when floor(UB) <= incumbent.

5. Algorithm: best-first branch and bound

```
sort items by v_i / m_i, descending
best <- 0;  frontier <- { (level 0, m=0, p=0, v=0) }
while frontier not empty:
    node <- extract max-UB node
    if floor(UB(node)) <= best: stop (heap) / skip
    k <- node.level + 1
    if m + m_k <= C and p + p_k <= P:      // include
        best <- max(best, v + v_k);  push child if floor(UB) > best
    push exclude child if floor(UB) > best
return best

UB = v + min( F_mass(k, C-m),  F_power(k, P-p) )
```

6. Hand trace on the instance (best-first)

Step	Node popped	UB	What happens
1-3	{}, {P1}, {P1,P2}	195.11	Dive down the greedy path
4	{P1,P2,P3} v=142	195.11	P4 would reach 16.6 kg: infeasible. Incumbent 160 later
5	{P1,P3}	188.94	Include P4: incumbent {P1,P3,P4} = 172. Exclude P4: UB 172 <= 172, pruned
6	{P1,P2,P4}	187.00	{P1,P2,P4,P5} = 172, ties; exclude branches pruned
7	{} (P1 excluded)	183.53	{P2} then {P2,P3} then {P2,P3,P4}: incumbent 174 (14.1 kg)
8	remaining nodes	<= 174	Heap top 174 <= incumbent: stop

Result: 47 nodes generated, 23 expanded, versus 64 complete selections (127 tree nodes).

7. Complexity analysis

O(n 2^n)	O(n^2)	~2,000
Worst-case time	Best-case time	Nodes at n = 20 (practical)

- Recurrence: T(n) = 2T(n-1) + O(n) gives O(n 2^n); the tree has at most 2^(n+1)-1 nodes and each bound is an O(n) scan.
- Space: DFS O(n); BFS and best-first up to O(2^n).
- Best case: the first greedy dive is optimal and everything else is pruned.
- Compare: DP is O(nW) = O(n x 150,000) at 0.1 g.

Proof of correctness

- Invariant: best = value of the best feasible selection seen; every discarded subtree has maximum value <= best.
- Initial: best = 0, nothing discarded.
- Maintenance: a node is discarded only if floor(UB) <= best. By the bound lemma every selection below it has value <= UB, and values are integers, so <= best. Infeasible children hold no feasible selections.
- Termination: the tree is finite and every node is expanded, pruned or a leaf, so at the end best is optimal.
- Q5: the min of two valid upper bounds is a valid upper bound, so the same proof holds.

8. Changed requirement: add a 35 W power limit

Set	Value	Mass	Power	Verdict
{P2,P3,P4} (old optimum)	174	14.1 kg	37 W	Infeasible
{P1,P3,P4}	172	13.4 kg	33 W	Optimal
{P1,P3,P5,P6}	172	14.4 kg	31 W	Optimal (tie)

- Technique still holds: add one feasibility test and use UB = min(mass bound, power bound); both are valid, so the minimum is valid.
- Best-first needs 39 nodes; DFS 51; BFS 45 (brute force confirms 172).
- DP would need a 2-D table (mass x power); greedy has no single ordering.

Q3 and Q4: greedy and search orders

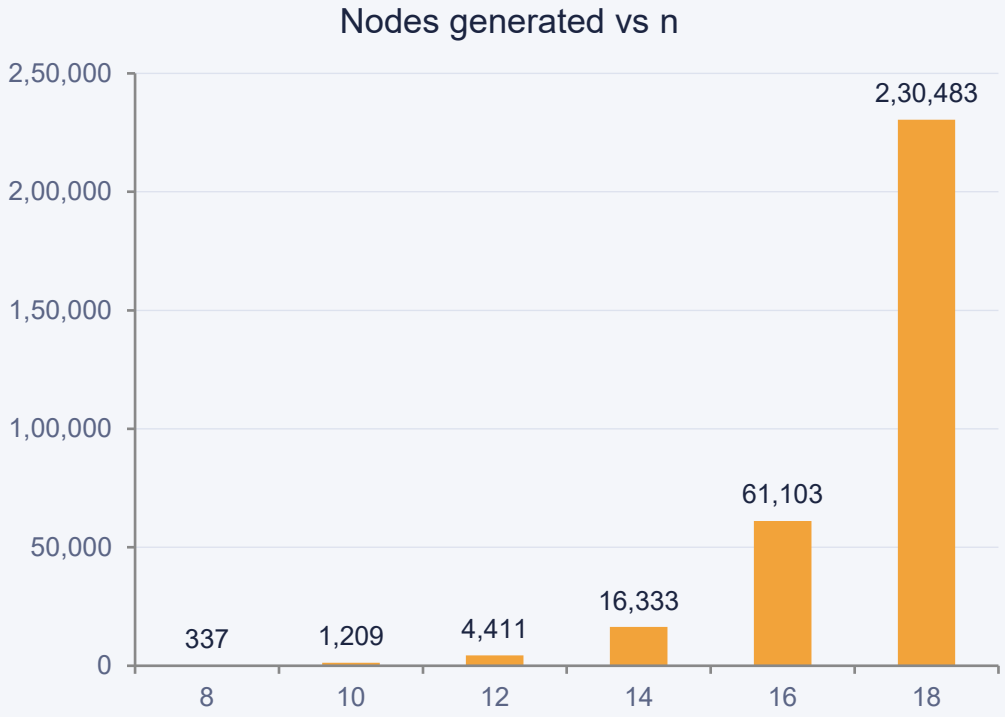
Teammate's greedy

- Stops at first misfit: {P1,P2,P3} = 142
- Skips misfits: + P5 = 160
- Optimum: 174
- Gap: 14 to 32 value units

Order	Generated	Expanded	Max frontier	Optimum found at
DFS (stack)	47	23	5	node 42
BFS (queue)	49	24	8	node 24
Best-first (heap)	47	23	6	node 36

BFS reaches the optimum first here, but cannot prove it until the end. Best-first stops as soon as the heap top cannot beat 174.

Worst-case input



- Construction: value = mass for every item (equal density), all masses EVEN, capacity ODD.
- Then UB = C at every node, but every reachable value is even, so best <= C - 1.
- floor(UB) > best always: no internal node is pruned.
- Result: about 44% of all 2^(n+1)-1 nodes at n = 18; nodes grow about 1.9x per extra item.

Real-world reference

References

- Martello and Toth, Knapsack Problems: Algorithms and Computer Implementations (Wiley, 1990): depth-first B&B with an LP-relaxation bound.
- Google OR-Tools knapsack solver (B&B and DP modes). Verify details in its docs.

How ours differs

- Theirs: integer weights, tighter bounds, item reductions, multi-dimensional support.
- MILP solvers add cuts and presolve.
- Ours: plain fractional bound, min over two resources, no reductions.

Code repository and team contributions

github.com/24i323-netizen/satellite-knapsack

Name	Pull request	Tests contributed
Karthikeya Prabu V K	#1 member1-dp	DP: selected items, empty items, zero capacity (3 tests)
Sree Ashwath S S	#2 member2-bnb-pr, #4 attribution	B&B: empty items, zero capacity, exact capacity, oversize item, power limit, feasibility (6 tests)
Vasantha Kumar M	#3 member3-greedy-pr	Greedy not optimal, capacity respected, all search orders find the optimum and are feasible, consistent statistics (6 tests)

- All four pull requests merged into main: 20 unit tests pass (5 original + 15 edge-case tests).
- Run python -m unittest discover -s tests, then the scripts in each member folder, to reproduce every number in this deck.

Summary

- Optimum: {P2,P3,P4}, value 174, 14.1 kg
- Branch and bound examines 47 of 127 tree nodes
- With 35 W: value 172, {P1,P3,P4}
- Greedy, backtracking, D&C and DP each fail for a concrete reason
- Worst case stays exponential; typical cases are tiny